2 - Implementation

Implement the shooting method with the following specifications:

(a) Use RK4 from HW2 as IVP solver.

(b) Use secant method as root finder.

(c) Report results in table, logging initial guesses s_0, s_1 for secant method, all intermediate values of s_0, s_1, s_2 for each iteration, the number of secant iterations used, and the final value of s obtained.

Use the following stopping conditions for the secant method:

$$|F(s_k)| < 10^{-10} \quad \text{and} \quad |s_k - s_{k-1}| < 10^{-10}$$

Also, add a reasonable maximum iteration cap.

Solution.

There are three main python files used for this implementation: 'rk4.py' for the RK4 method, 'secant.py' for the secant method, and 'hw3.py' for the main script that runs the shooting method and generates tables of results.

Below is the the RK4 method from HW2 that has been updated to take an additional parameter, F (the function defining the ODE system), as well as updated default parameters ($h = 0.01,\ T = \frac{\pi}{2}$).

Listing 1: Updated RK4 Implementation (rk4.py)

```python
import numpy as np

def RK4_step(F, y, h):
    k1 = F(y)
    k2 = F(y + (h / 2) * k1)
    k3 = F(y + (h / 2) * k2)
    k4 = F(y + h * k3)

    y_new = y + (h / 6) * (k1 + 2 * k2 + 2 * k3 + k4)
    return y_new

def RK4_solver(F, y_0, h=0.01, T=np.pi/2):
    num_steps = int(T / h)
    # Adjust h to ensure we end exactly at T
    h = T / num_steps
    y_values = np.zeros((num_steps + 1, len(y_0)))
    y_values[0] = y_0
```

```python

    for n in range(num_steps):
        y_values[n + 1] = RK4_step(F, y_values[n], h)

    return y_values
```

First we need to determine appropriate initial guesses for the secant method. We can do this by solving the IVP with two different initial slopes and checking the sign of the resulting residuals. The following code was ran to determine the initial guesses:

Listing 2: Determining Initial Guesses for Secant Method (hw3.py)

```python
from rk4 import RK4_solver
import numpy as np

Y_0 = 0
Y_RIGHT_BC = 2

def F(u):
    y, z = u
    return np.array([z, -y])

# residual function is F(s)
def residual(s, h=0.01):
    u_0 = np.array([Y_0, s])
    y_values = RK4_solver(F, u_0, h=h)

    # Return F(s) = y(T) - Y_RIGHT_BC
    return y_values[-1, 0] - Y_RIGHT_BC

if __name__ == "__main__":
    s_values = [4 / np.pi, 4/ np.pi + 1]
    for s in s_values:
        print(f"s = {s}, F(s) = {residual(s)}")
```

Our initial choice of $s_0 = \frac{4}{\pi}$ had a residual of $F(s_0) \approx -0.7267604552662292$, and our second choice of $s_1 = \frac{4}{\pi} + 1$ had a residual of $F(s_1) \approx 0.2732395447326774$. Since the residuals have opposite signs, we can proceed with these as our initial guesses for the secant method.

Below is the implementation of the secant method:

Listing 3: Secant Method (secant.py)

```python
def secant_method(F, s0, s1, h=0.01, tol=1e-10, max_iter=100):
    # Notation
    '''
    s_past = s_{n-1} [initial guess s0]
    s_current = s_n [initial guess s1]
```

```
    f_past = F(s_{n-1}) = F(s_past)
    f_current = F(s_n) = F(s_current)
    s_next = s_{n+1} [uses secant formula]
    '''
    # Keep track of all s values for table
    s_values = [s0, s1]

    s_past, s_current = s0, s1
    F_past = F(s_past, h=h)
    F_current = F(s_current, h=h)

    # Keep track of F(s) values
    F_values = [F_past, F_current]

    for _ in range(max_iter):
        # Use secant formula to find the next time step
        s_next = s_current - F_current * (s_current - s_past) / (F_current - F_past)

        F_next = F(s_next, h=h)

        s_values.append(s_next)
        F_values.append(F_next)

        # Check for convergence
        if abs(F_next) < tol and abs(s_next - s_current) < tol:
            return s_values, F_values

        # Update for the next iteration
        s_past, s_current = s_current, s_next
        F_past, F_current = F_current, F_next


    # If we reach here, it means we did not converge within the maximum iterations
    print("Warning: Maximum iterations reached without convergence.")
    return s_values, F_values
```

For implementation of the full shooting method, we use the ‘residual‘ function defined in ‘hw3.py‘ as our $F(s)$ for the secant method. The full implementation of the shooting method is as follows:

Listing 4: Full Shooting Method Implementation (hw3.py)

```

from secant import secant_method

# Used to generate tables of results for secant method iterations
def generate_tables(s_values, F_values, path):
```

```python
    table_line = r"\hline" + "\n"
    with open(path, "w") as file:
        # Add final s value and F(s)
        # Determine final index for s and F(s) values
        n_final = len(s_values) - 1
        total_iterations = n_final - 1
        file.write(f"Total iterations: {total_iterations}\n")
        file.write(f"$s_{{{n_final}}}$: {s_values[-1]:.6f}\n")
        file.write(f"$F(s_{{{n_final}}})$: {F_values[-1]:.6e}\n\n")

        # First table with inital s values and their corresponding F(s) values
        # n | s_n | F(s_n)
        file.write( r"\begin{center}" + "\n")
        file.write(r"\begin{tabular}{|c|c|c|}" + "\n")
        file.write(table_line)
        file.write(r"$n$ & $s_n$ & $F(s_n)$ \\" + "\n")
        file.write(table_line)

        for i in range(2):
            s, F_s = s_values[i], F_values[i]
            file.write(rf"${i}$ & ${s:.6f}$ & ${F_s:.6e}$ \\" + "\n")
            file.write(table_line)

        file.write(r"\end{tabular}" + "\n")
        file.write(r"\end{center}" + "\n")

        file.write("\n\n") # Add some space between the tables

        # Second table with secant method iterations
        # n | s_{n-1} | s_n | s_{n+1} | F(s_{n+1})
        file.write(r"\begin{center}" + "\n")
        file.write(r"\begin{tabular}{|c|c|c|c|c|}" + "\n")
        file.write(table_line)
        file.write(r"$n$ & $s_{n-1}$ & $s_n$ & $s_{n+1}$ & $F(s_{n+1})$ \\" + "\n")
        file.write(table_line)

        for n in range(1, len(s_values) - 1):
            s_past, s_current, s_next = s_values[n-1], s_values[n], s_values[n + 1]
            F_next = F_values[n + 1]
            file.write(
                rf"${n}$ & ${s_past:.6f}$ & ${s_current:.6f}$ & ${s_next:.6f}$ & ${F_next:.6e}$ \\" + "\n"
            )
            file.write(table_line)
        file.write(r"\end{tabular}" + "\n")
```

```
        file.write(r"\end{center}" + "\n")
    return

if __name__ == "__main__":
    # Continues from previous code for determining initial guesses.
    s0, s1 =  4 / np.pi, 4/ np.pi + 1
    s_values, F_values = secant_method(residual, s0, s1)
    generate_tables(s_values, F_values, "./outputs/tables.txt")
```

The following tables were generated:

n	s_n	$F(s_n)$
0	1.273240	$-7.267605e-01$
1	2.273240	$2.732395e-01$

n	s_{n-1}	s_n	s_{n+1}	$F(s_{n+1})$
1	1.273240	2.273240	2.000000	$-1.554312e-15$
2	2.273240	2.000000	2.000000	$4.440892e-16$

A total of 2 iterations were used to converge to the final value of $s \approx 2$ with a final residual of $F(s) \approx 4.440892e-16$.

---

3 - Two Independent Verification Checks

After obtaining s and the numerical solution $y_{N+1}(x)$, verify correctness of your implementation with the following checks:

(i) Check the residual F by reporting the final residual magnitude $|F(s)|$.

(ii) Check solution error by comparing $y_{N+1}(x)$ to the exact solution $y(x)$. Compute the L^2 error:

$$e_{L^2} = \sum_i (y_i(x_i) - y(x_i))^2$$

Solution.

From our results in question 2, the magnitude of our residual is on the order of machine precision 10^{-16}, which indicates that our implementation is correctly finding a root of the residual function $F(s)$.

For the L^2 error check, the following code was added to 'hw3.py':

Listing 5: L2 Error Implementation (hw3.py)

```
def exact_solution(x):
```

```
    return 2 * np.sin(x)

if __name__ == "__main__":
    # Continues from previous code for secant method and table generation.
    # L2 Error Check
    s_final = s_values[-1]
    h = 0.01
    u_0 = np.array([Y_0, s_final])
    y_values = RK4_solver(F, u_0, h=h)
    x_values = np.linspace(0, np.pi / 2, len(y_values))
    y_exact = exact_solution(x_values)
    l2_error = np.sqrt(np.sum((y_values[:, 0] - y_exact) ** 2))
    print(f"L2 error between numerical and exact solution: {l2_error:.6e}")
```

It was found that the L^2 error was approximately $8.332945e-10$, which indicates that our numerical solution is very close to the exact solution.

---

4 - Step Size Refinement and Order of Accuracy

Perform a refinement study using the following step sizes:

$$h = \left\{ \frac{\pi}{10}, \frac{\pi}{20}, \frac{\pi}{40}, \frac{\pi}{80}, \frac{\pi}{160} \right\}$$

For each h, run the full shooting procedure and compute the L^2 error and residual.

(a) Create a table with the following columns:

$$h, \quad s, \quad |F(s)|, \quad e_{L^2}(h)$$

(b) For $h = \frac{\pi}{160}$, plot y_i and $y(x)$ on the same graph.

(c) Make a log-log plot of e_{L^2} vs h. Estimate the observed order of accuracy.

Solution.

The following code was added to ‘hw3.py‘ to perform the refinement study and generate the required tables and plots:

Listing 6: Step Size Refinement Study (hw3.py)

```
import matplotlib.pyplot as plt

if __name__ == "__main__":
```

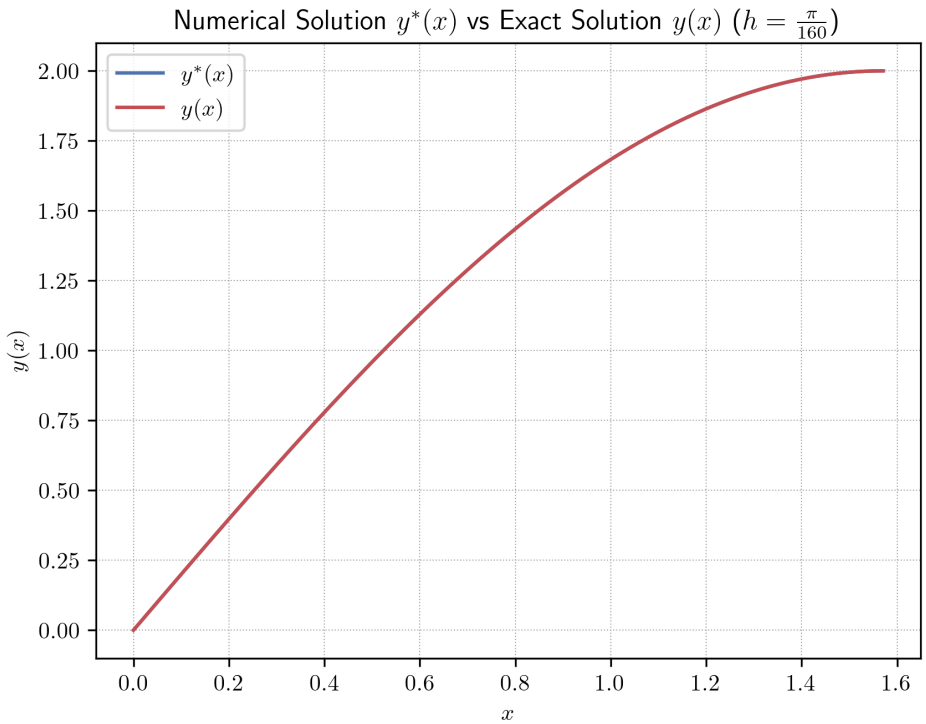
```python
    # ... Previous code ...
    # ----------------------------------------
    # (a) Generate table for step size refinement
    # ----------------------------------------
    s0, s1 =  4 / np.pi, 4/ np.pi + 1
    pi = np.pi
    h_values = [pi/10, pi/20, pi/40, pi/80]
    s_values = []
    F_values = []
    l2_values = []

    # Compute final s, F(s), and L2 error for each h
    for h in h_values:
        s_values, F_values = secant_method(residual, s0, s1, h=h)
        s_final = s_values[-1]
        u_0 = np.array([Y_0, s_final])
        y_values = RK4_solver(F, u_0, h=h)
        x_values = np.linspace(0, pi/2, len(y_values))
        y_exact = exact_solution(x_values)
        l2_error = np.sqrt(np.sum((y_values[:, 0] - y_exact) ** 2))

        s_values.append(s_final)
        F_values.append(F_values[-1])
        l2_values.append(l2_error)

    # Write table to file
    # h | s_final | F(s_final) | L2 Error
    with open("./outputs/step_size_study.txt", "w") as file:
        table_line = r"\hline" + "\n"

        file.write(r"\begin{center}" + "\n")
        file.write(r"\begin{tabular}{|c|c|c|c|}" + "\n")
        file.write(table_line)
        file.write(r"$h$ & $s$ & $|F(s)|$ & $e_{L^2}(h)$ \\" + "\n")
        file.write(table_line)

        for h, s, F_final, l2 in zip(h_values, s_values, F_values, l2_values):
            F_final = abs(F_final)
            file.write(
                rf"${h:.6f}$ & ${s:.6f}$ & ${F_final:.6e}$ & ${l2:.6e}$ \\" + "\n"
            )
            file.write(table_line)

        file.write(r"\end{tabular}" + "\n")
        file.write(r"\end{center}" + "\n")

```

```python
    # ... Latex and Styling ...
    # Get list of default colors for style
    prop_cycle = plt.rcParams["axes.prop_cycle"]
    default_colors = prop_cycle.by_key()["color"]

    # ----------------------------------------
    # (b) Plot y* and y for h = pi/160
    # ----------------------------------------
    h = pi/160
    s_values, F_values = secant_method(residual, s0, s1, h=h)
    s_final = s_values[-1]
    u_0 = np.array([Y_0, s_final])
    y_values = RK4_solver(F, u_0, h=h)
    x_values = np.linspace(0, pi/2, len(y_values))
    y_exact = exact_solution(x_values)

    plt.plot(x_values, y_values[:, 0], label=r"$y^*(x)$", color=default_colors[0])
    plt.plot(x_values, y_exact, label=r"$y(x)$", color=default_colors[2])
    plt.xlabel(r"$x$")
    plt.ylabel(r"$y(x)$")
    plt.title(r"Numerical Solution $y^*(x)$ vs Exact Solution $y(x)$")
    plt.legend()
    plt.savefig("./outputs/comparison_graph.png", dpi=300)
    plt.clf()

    # ----------------------------------------
    # (c) plot log-log graph of L2 error vs h
    # ----------------------------------------
    plt.loglog(h_values, l2_values, marker="o")
    plt.xlabel(r"$h$")
    plt.ylabel(r"$e_{L^2}(h)$")
    plt.title(r"Log-Log Plot of $L^2$ Error vs Step Size $h$")
    plt.savefig("./outputs/error_graph.png", dpi=300)
```

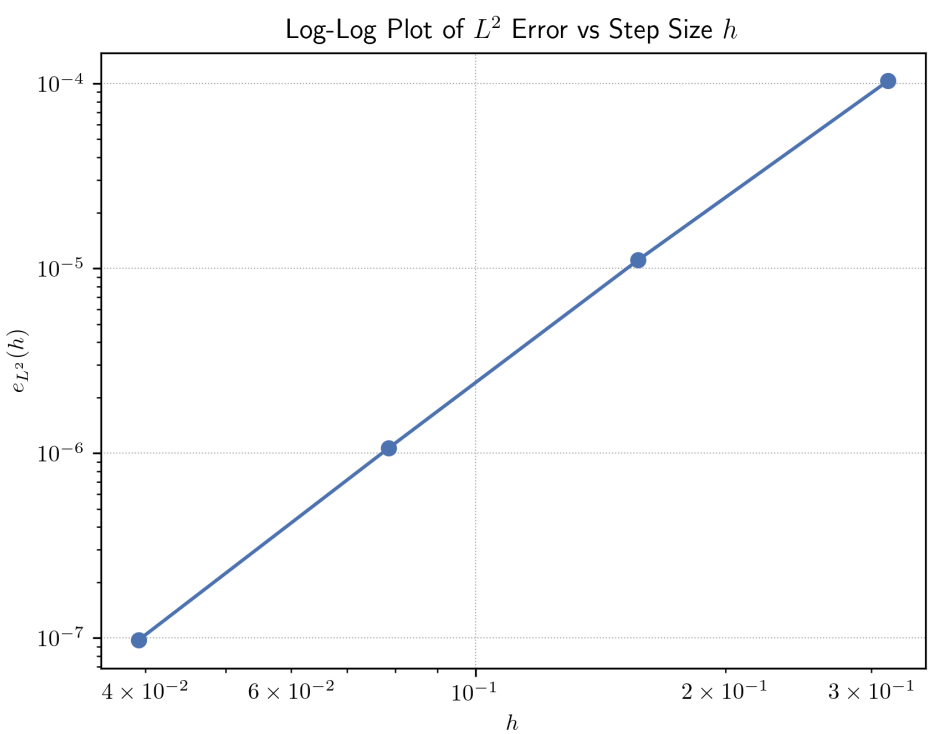
(A) Step Size Refinement Table

h	s	$\lvert F(s)\rvert$	$e_{L^2}(h)$
0.314159	1.273240	$7.267605e-01$	$1.035517e-04$
0.157080	2.273240	$2.732395e-01$	$1.111394e-05$
0.078540	2.000000	$4.440892e-16$	$1.062848e-06$
0.039270	2.000000	$0.000000e+00$	$9.735252e-08$

(B) Numerical vs. Exact Solution for $h = \frac{\pi}{160}$



(C) Log-Log Plot of L^2 Error vs Step Size h



The order of accuracy is theoretically $O(h^4)$ from RK4.

We observe that as we refine the step size h, the L^2 error decreases by about a magnitude of 10 each time h is halved. In particular, this ratio increases as h decreases. Also, the table shows that the shooting error $|F(s)|$ decreases very rapidly as h decreases. This indicates that shooting error plays a larger role for larger h values. So, as we refine h, the error becomes dominated by the RK4 error, which is $O(h^4)$. We don't observe a perfect $O(h^4)$ relationship due to other sources of error (such as discretization error and machine precision), but we do see a clear trend of rapidly decreasing error as h decreases, which is consistent with the expected order of accuracy.

Our observations also match the results of the log-log plot, where we observe a linear, and therefore exponential, decrease in error as h decreases.

In our comparison graph, we see that the numerical solution closely matches the exact solution, which is consistent with our low L^2 error value.